



Research Intake 2026

Day 31

Object Detection Fundamentals

A Beginner's Guide for Students

Learning Computer Vision Through Images, Videos, and Artificial Intelligence

Gudsky Research Foundation

Table of Contents

1 From Classification to Detection	4
1.1 Why Detection Is Harder Than Classification	4
1.2 Detection = Backbone + Detection Head	4
1.3 This Day's Practical Project: PP11	5
1.4 A Note on Terminology: Detection vs. Localization vs. Recognition	5
2 Bounding Boxes & Localization	6
2.1 Bounding Box Representations	6
2.2 Ground Truth Annotation Formats	6
2.3 Intersection over Union: The Core Geometric Primitive	7
3 Detection Evaluation Metrics.....	8
3.1 Precision and Recall at an IoU Threshold	8
3.2 Average Precision (AP) per Class	9
3.3 Mean Average Precision (mAP).....	9
3.4 AP@0.5 vs. AP@[0.5:0.95]	10
3.5 Worked mAP Calculation.....	10
4 Two-Stage Detectors: R-CNN Family	10
4.1 R-CNN's Original Region-Proposal Pipeline	11
4.2 Fast R-CNN's Shared Feature Computation	12
4.3 Faster R-CNN's Learned Region Proposal Network	12
4.4 Why Two-Stage Detectors Trade Speed for Accuracy	12
5 One-Stage Detectors: YOLO	13
5.1 YOLO's Core Insight: Detection as Single Regression	13
5.2 Grid-Cell Prediction Mechanics	14
5.3 Evolution Across YOLO Versions	14
5.4 Why YOLO Became the Real-Time Default.....	15
6 One-Stage Detectors: SSD.....	15
6.1 SSD's Core Insight: Multi-Scale Feature Maps	16
6.2 Why Multi-Scale Detection Matters for Small Objects	16
6.3 Default Boxes: SSD's Version of Anchors	17
6.4 SSD vs. YOLO: A Direct Comparison.....	17
7 Anchor Boxes & Non-Max Suppression	18
7.1 Anchor Boxes as Priors for Scale and Aspect Ratio	18
7.2 Anchor Design Trade-offs	19
7.3 Non-Max Suppression: Mechanics	20
7.4 Soft-NMS: A Refinement.....	21
8 Two-Stage vs. One-Stage: Tradeoffs	22
8.1 Synthesising Sections 4 Through 6.....	22
8.2 Expected Performance Differences	23

8.3 A Fair Head-to-Head Evaluation Methodology.....	23
8.4 Closing Perspective: What PP11 Should Reveal.....	24
9 Real-World Applications.....	24
9.1 Autonomous Driving Perception.....	24
9.2 Retail and Inventory Monitoring.....	25
9.3 Wildlife Camera-Trap Analysis.....	25
9.4 Security and Surveillance: An Ethical Callback.....	26
10 Research Frontiers.....	26
10.1 Transformer-Based Detection: DETR.....	26
10.2 Open-Vocabulary Detection.....	27
10.3 Forward References: Day 32 and Day 33.....	27
10.4 Closing Perspective: The Detection Lineage in Context.....	27
11 Common Pitfalls: A Practical Debugging Checklist.....	28
11.1 A Note on Reproducibility.....	29
11.2 A Note on Comparing Against Published Benchmarks.....	29
Glossary of Terms.....	30
Chapter Summary: Key Takeaways.....	30
Assignment / Practical Project Brief.....	32
A Core Requirements.....	32
B Dataset.....	32
C Deliverables.....	33
D Grading Rubric.....	34
E Tips and Common Pitfalls.....	34
F Submission Checklist.....	35

The future of AI is bright, and you can be part of it!

1 From Classification to Detection

Day 26 §4 introduced the computer vision task taxonomy — classification, localization, detection, segmentation, and beyond — as the map for this entire module. This day picks up the third row of that taxonomy directly: object detection, the task of identifying every object of interest in an image and reporting both what it is and where it is, for potentially many objects simultaneously.

1.1 Why Detection Is Harder Than Classification

Image classification (Day 29’s running example throughout) assumes a single label describes the entire image. Object detection removes that assumption entirely, and the removal introduces genuine new difficulty along three axes. The number of objects in an image is not known in advance — a photograph might contain zero objects of interest, one, or dozens, and a detector must handle all of these cases without being told the count beforehand. The location of each object is not known in advance either — a detector must search the image, implicitly or explicitly, for where objects might be, rather than simply describing a fixed input. And class and location must be predicted jointly, per object — it is not enough to separately know "there is a dog somewhere" and "there is a box somewhere"; the detector must correctly pair each class label with its corresponding location. Every architecture this day covers is, fundamentally, a different strategy for handling these three axes of difficulty simultaneously.

1.2 Detection = Backbone + Detection Head

Every object detector this day covers, without exception, follows the same high-level structural pattern: a backbone network extracts features from the input image, and a detection head — a task-specific set of layers appended to the backbone — transforms those features into the final set of predicted boxes and class labels. This is precisely why Day 30’s entire backbone-architecture survey was necessary background for this day: the backbone choice (ResNet, EfficientNet, or a Swin-style hybrid, per Day 30 §9.4’s direct forward reference) is a genuine, consequential design decision for any detector, and Day 30’s decision framework (dataset size, latency budget, deployment target) applies to choosing a detection backbone exactly as it applied to choosing a classification backbone. What differs between the detectors this day covers is entirely the detection head — how the backbone’s extracted features are turned into boxes and classes — and Sections 4 through 6 develop three structurally different answers to that question.

 Classification vs. Detection, Made Concrete

A classifier looking at a photograph of a street scene outputs a single label, perhaps "street scene" or, if forced into a narrower single-label framing, whichever object is most prominent — "car," say.

A detector looking at the same photograph outputs a list: perhaps three cars (each with its own bounding box and confidence score), two pedestrians, and a traffic light — exactly the kind of structured, multi-object output an autonomous vehicle’s perception system (Section 9) genuinely needs, and that a single whole-image label could never provide.

Property	Classification (Day 29)	Object Detection (this day)
----------	-------------------------	-----------------------------

Output structure	One label per image	A variable-length list of (box, label, score) triples
Number of objects	Assumed to be exactly one	Unknown in advance; zero, one, or many
Spatial information	None	A bounding box per detected object
Typical loss function	Cross-entropy (Day 29 §7.2)	A combined localisation + classification loss
Evaluation metric	Accuracy / top-k accuracy	mAP (Section 3)



Worked Example: Backbone vs. Head: Where the Parameters Actually Live

A typical Faster R-CNN with a ResNet-50 backbone (Day 30 §4.3) has roughly 41 million total parameters — approximately 23 million in the backbone alone (unchanged from Day 30's ResNet-50 classification network, minus its final classification layer), with the remaining ≈ 18 million split between the RPN (§4.3) and the final detection head's box-regression and classification layers.

This roughly 55/45 split between backbone and head parameters is a useful sanity check when reasoning about where to focus optimisation effort: swapping in a more efficient backbone (Day 30 §6's MobileNet or EfficientNet) directly reduces the larger of these two components, while detection-head-specific optimisations (fewer anchor boxes, §7.2; a lighter-weight head design) address the smaller remaining share — both matter, but the backbone choice, exactly as Section 1.2 argues, is usually the higher-leverage decision for overall model size and speed.

1.3 This Day's Practical Project: PP11

This day carries a graded assignment, PP11, developed in full at the end of this compendium. Unlike the theory-only days that surround it, PP11 requires fine-tuning both a YOLO model and an SSD model on the same dataset and directly comparing them — making Sections 5, 6, and 8 in particular directly load-bearing for the assignment, not merely conceptual background. It is worth reading this day with that assignment in mind from the outset, since several sections are structured specifically to prepare you for the comparison PP11 requires.

1.4 A Note on Terminology: Detection vs. Localization vs. Recognition

These three terms are sometimes used loosely or interchangeably in casual conversation, but this compendium uses them precisely, matching Day 26 §4's taxonomy. Localization refers specifically to the single-object case: given an image known to contain exactly one object of interest, predict its bounding box (and typically its class). Detection, this day's subject, generalises localization to the multi-object case: an unknown number of objects, each requiring its own box and class. Recognition is a broader, less precisely defined term sometimes used to encompass classification, detection, and related tasks collectively — worth flagging specifically because it appears in some older literature and in adjacent fields (biometrics, for instance) with a meaning that does not map cleanly onto this compendium's more precise taxonomy, and is worth clarifying by context rather than assuming a fixed definition when encountered elsewhere.

2 Bounding Boxes & Localization

2.1 Bounding Box Representations

A bounding box is typically represented in one of two equivalent coordinate formats. The `xyxy` format stores the top-left and bottom-right corner coordinates directly: `(x_min, y_min, x_max, y_max)`. The `xywh` format instead stores a centre point and dimensions: `(x_center, y_center, width, height)`. Both formats describe the same rectangle and convert losslessly between each other, but different detection frameworks and datasets default to different formats — YOLO's native label format uses `xywh` (typically normalised to `[0,1]` relative to image dimensions), while many general-purpose datasets and libraries default to `xyxy` — and mixing the two formats without an explicit, correct conversion is a common, easy-to-make bug when combining code or datasets from different sources.

It is worth pausing on why this seemingly mundane representational choice matters enough to open this section with. Every downstream computation this day develops — IoU (Section 2.3), anchor matching (Section 7.1), NMS (Section 7.3), and the entire evaluation pipeline (Section 3) — operates directly on box coordinates, and a silent format mismatch propagates incorrect values through every one of these downstream computations without ever raising an explicit error, exactly the kind of bug Day 27 §1.1's "garbage in, garbage out" framing warned about in a preprocessing context and that recurs here in a geometric, rather than a pixel-value, form.

2.2 Ground Truth Annotation Formats

Several annotation format conventions are worth recognising, since PP11's dataset will arrive in one of them. COCO format stores annotations in a single JSON file, with bounding boxes in `(x_min, y_min, width, height)` — notice this is neither pure `xyxy` nor pure `xywh`, but a specific hybrid worth double-checking against library documentation rather than assuming. Pascal VOC format stores one XML file per image, with boxes in `xyxy` format. YOLO's `txt` format stores one plain-text file per image, one line per object, with normalised `xywh` coordinates and no file-level structure at all beyond this per-line convention. Correctly parsing whichever format PP11's dataset uses, and converting it consistently to whatever format the chosen detection framework expects internally, is a genuine first practical step in the assignment, worth verifying carefully before any training begins.

A related practical detail worth flagging while on the subject of annotation formats: images are frequently resized during preprocessing (Day 27 §2.1) before being fed to a detector, and bounding box coordinates must be rescaled by the identical factor applied to the image itself — a box recorded relative to a 1920×1080 original image is meaningless, or actively wrong, if applied unchanged to a 640×360 resized version without a corresponding coordinate rescaling. Most detection framework data loaders handle this rescaling automatically as part of their standard augmentation pipeline, but it is worth verifying explicitly (by visualising a resized image with its rescaled boxes overlaid, per Section 11's debugging checklist) rather than assuming, particularly when writing any custom data loading code for PP11's specific dataset.

2.3 Intersection over Union: The Core Geometric Primitive

Intersection over Union (IoU), illustrated in Figure 1, is the single geometric primitive that everything else in this day — evaluation metrics (Section 3), anchor box matching (Section 7), and non-max suppression (Section 7) — builds directly on top of. IoU measures how well two boxes overlap: the area of their intersection divided by the area of their union, producing a single score between 0 (no overlap at all) and 1 (perfect overlap).

Intersection over Union (IoU): Measuring Box Overlap

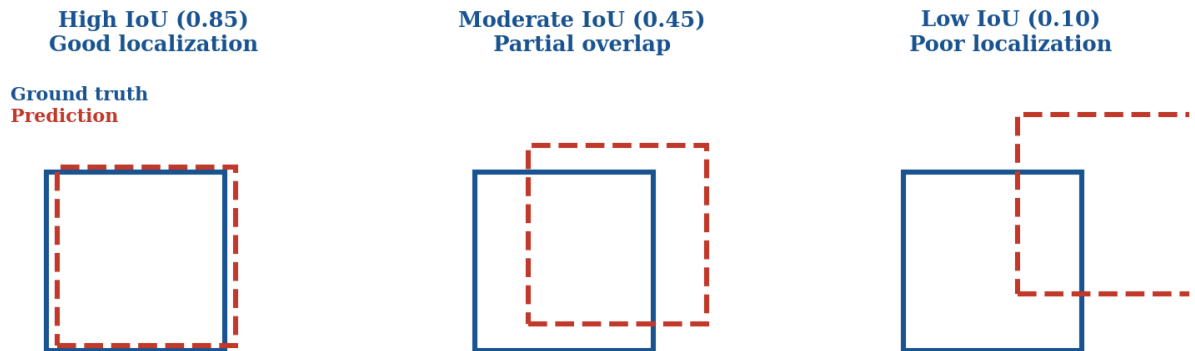


Figure 1. Intersection over Union (IoU) measures bounding box overlap on a scale from 0 to 1, forming the geometric basis for detection evaluation, anchor matching, and non-max suppression throughout this day.

Worked Example: Computing IoU by Hand

A ground-truth box spans $x: [2,7]$, $y: [2,7]$ (a 5×5 square, area 25). A predicted box spans $x: [4,9]$, $y: [3,8]$ (also 5×5 , area 25).

Intersection: the overlapping region spans $x: [4,7]$, $y: [3,7]$ — a 3×4 rectangle, area 12.

Union: $25 + 25 - 12 = 38$.

IoU: $12 / 38 \approx 0.316$.

By the common $\text{IoU} \geq 0.5$ convention (Section 3.1), this prediction would NOT count as a correct match despite genuinely overlapping the ground truth — the same "close but not close enough" outcome Day 28 §5.3's RANSAC worked example encountered with a different geometric threshold, underscoring how consequential a seemingly small threshold choice can be.

```
def compute_iou(box_a, box_b):
    # boxes in xyxy format (§2.1): (x_min, y_min, x_max, y_max)
    x1 = max(box_a[0], box_b[0])
    y1 = max(box_a[1], box_b[1])
    x2 = min(box_a[2], box_b[2])
    y2 = min(box_a[3], box_b[3])

    intersection = max(0, x2 - x1) * max(0, y2 - y1)
    area_a = (box_a[2] - box_a[0]) * (box_a[3] - box_a[1])
    area_b = (box_b[2] - box_b[0]) * (box_b[3] - box_b[1])
    union = area_a + area_b - intersection
```

```
return intersection / union if union > 0 else 0.0
```

3 Detection Evaluation Metrics

3.1 Precision and Recall at an IoU Threshold

A detection is counted as a true positive only if it satisfies two conditions simultaneously: the predicted class matches the ground truth, and the predicted box's IoU (Section 2.3) against the ground truth box exceeds a chosen threshold — conventionally 0.5, though stricter thresholds are common in more demanding benchmarks. Precision then measures what fraction of the detector's predictions were correct (true positives divided by all predictions made), while recall measures what fraction of the actual ground-truth objects were successfully detected (true positives divided by all ground-truth objects) — the same precision/recall definitions Day 27 §9.2's pitfalls table used in a preprocessing context, now applied to detection specifically.

Worked Example: Distinguishing a Classification Error from a Localization Error

A detector predicts a box with 0.9 confidence, correctly classified as "dog," but with an IoU of only 0.3 against the true dog's ground-truth box.

Under the standard true-positive criterion (§3.1): this prediction is a FALSE POSITIVE, despite the class label being entirely correct — IoU 0.3 falls below the conventional 0.5 threshold, so the localization is judged too imprecise to count as a correct detection.

This distinction matters directly for interpreting PP11's results: a detector with a surprisingly low mAP might be classifying objects correctly but localising them poorly (a Section 4–7 architectural or training issue), rather than confusing object classes outright (a different kind of problem entirely) — visualising predicted boxes directly (per §11's debugging checklist) is what distinguishes these two very different failure modes, since the mAP number alone does not.

Outcome	Condition	Contributes To
True Positive (TP)	Correct class AND $\text{IoU} \geq \text{threshold}$	Precision (numerator) and Recall (numerator)
False Positive (FP)	Prediction with no matching ground truth (wrong class or IoU too low)	Precision (denominator only)
False Negative (FN)	A ground-truth object with no matching prediction at all	Recall (denominator only)

3.2 Average Precision (AP) per Class

A detector's predictions each carry a confidence score, and adjusting the confidence threshold used to decide "is this prediction kept or discarded" trades precision against recall directly — a very low threshold keeps nearly every prediction (high recall, but low precision, since many low-confidence false positives are

kept), while a very high threshold keeps only the most confident predictions (high precision, but low recall, since some genuine objects are missed). Average Precision (AP) summarises a detector's performance across this entire trade-off in a single number: it is the area under the precision-recall curve, illustrated in Figure 2, produced by sweeping the confidence threshold from 0 to 1 and plotting the resulting precision and recall at every threshold.

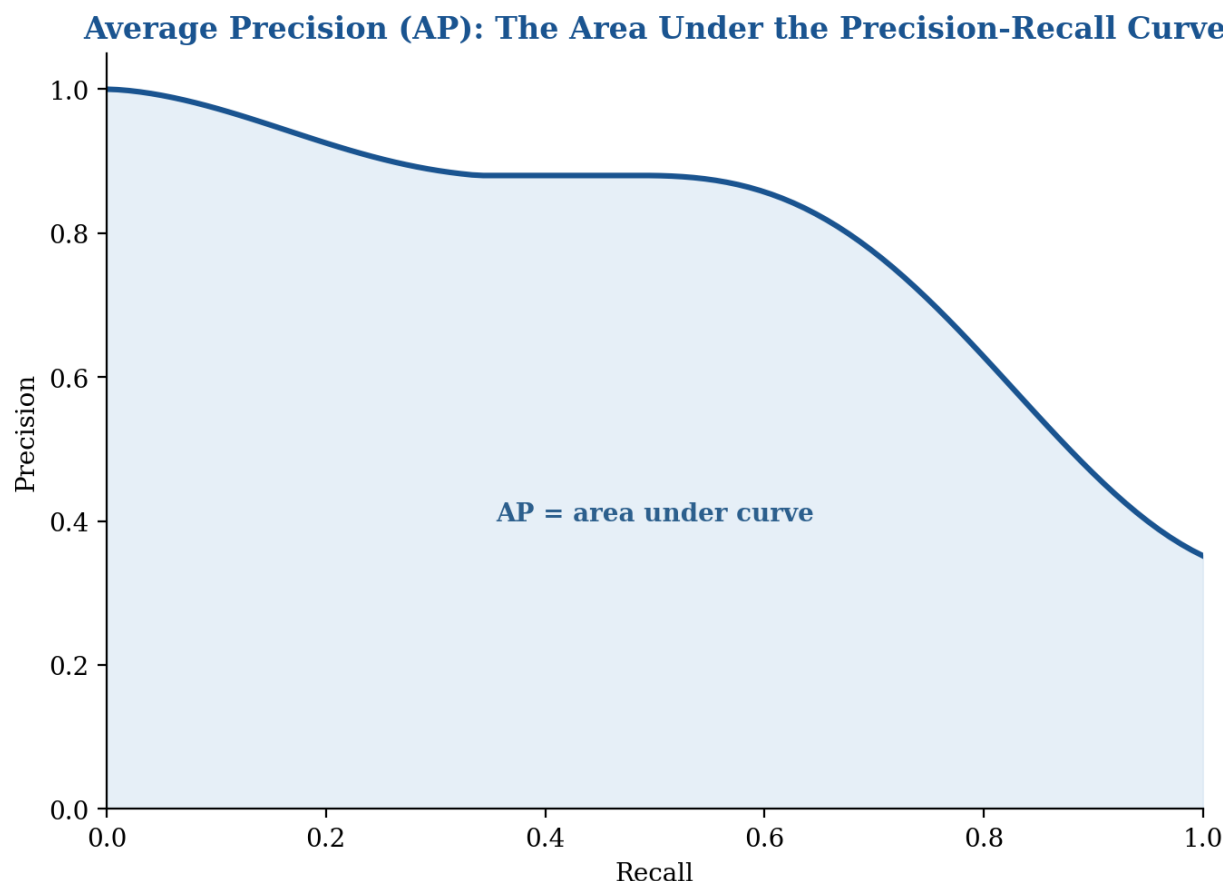


Figure 2. Average Precision (AP) is the area under the precision-recall curve, summarising detector performance across every possible confidence threshold in a single number.

3.3 Mean Average Precision (mAP)

AP (Section 3.2) is computed separately for each object class, since a detector might be excellent at detecting cars but poor at detecting bicycles, and a single combined metric would obscure this difference. Mean Average Precision (mAP) averages AP across all classes in the dataset, producing the single-number summary metric this entire day's comparisons — and PP11's required comparison specifically — will report and rely on.

3.4 AP@0.5 vs. AP@[0.5:0.95]

Two conventions for reporting AP are worth distinguishing explicitly, since published benchmark numbers are meaningless without knowing which convention was used. AP@0.5 (the Pascal VOC convention) uses

a single, comparatively lenient IoU threshold of 0.5 to decide true positives (Section 3.1). $AP@[0.5:0.95]$ (the COCO convention, and the field's current de facto standard for serious benchmark reporting) computes AP at ten different IoU thresholds (0.5, 0.55, 0.60, ..., 0.95) and averages across all ten, producing a stricter, more comprehensive metric that rewards precise localization rather than merely "roughly correct" localization. Day 26 §8's brief mention of mAP promised a deep dive here — this is where that dive happens, and it is worth remembering for PP11 that both conventions should ideally be reported, or at minimum that whichever convention is used should be stated explicitly and applied identically to both YOLO and SSD for a fair comparison.

3.5 Worked mAP Calculation

Worked Example: A Small, Complete AP Calculation

Suppose a detector makes 5 predictions for a single class, ranked by confidence, with ground truth containing 3 true objects of that class. After matching each prediction to ground truth via $IoU \geq 0.5$ (Section 3.1): predictions 1, 2, and 4 are true positives (TP); predictions 3 and 5 are false positives (FP).

Precision/recall at each rank: after pred 1 (TP): precision=1/1=1.0, recall=1/3≈0.33. After pred 2 (TP): precision=2/2=1.0, recall=2/3≈0.67. After pred 3 (FP): precision=2/3≈0.67, recall=2/3≈0.67. After pred 4 (TP): precision=3/4=0.75, recall=3/3=1.0. After pred 5 (FP): precision=3/5=0.6, recall=1.0.

AP (via the interpolated precision-recall envelope, Section 3.2): taking the maximum precision achieved at or beyond each recall level and integrating gives an AP of approximately 0.89 for this class — the small worked numbers here are deliberately simple, but the mechanical process (rank predictions, match against ground truth, trace out precision and recall, integrate) is identical to what a real evaluation script performs across thousands of predictions.

This same computation, repeated per class and averaged, is exactly the mAP number PP11 requires you to compute and report for both your YOLO and SSD models — using a library implementation (Section 8.2's worked comparison uses standard COCO evaluation tooling) rather than a manual calculation for the actual assignment, but understanding this manual process is what makes the library's output number interpretable rather than a mysterious black box.

4 Two-Stage Detectors: R-CNN Family

This section covers the R-CNN family in roughly chronological, problem-response order — each successive model addressing a specific, identified bottleneck in its predecessor — the same incremental refinement pattern Day 28 §1.4 traced for classical feature detection and Day 30 §1.1–§1.2 traced for backbone architectures. Recognising this pattern here, rather than treating R-CNN, Fast R-CNN, and Faster R-CNN as three unrelated named models to memorise independently, makes each architectural change's motivation considerably easier to understand and retain.

4.1 R-CNN's Original Region-Proposal Pipeline

R-CNN (Regions with CNN features, Girshick et al., 2014) was the first detector to successfully combine classical region proposals with deep, learned features — a direct, concrete instance of Day 26 §6's "deep

learning displaces handcrafted methods" theme, applied specifically to detection. R-CNN's pipeline, shown in Figure 3's top row, proceeds in four stages: Selective Search (a classical, non-learned algorithm producing roughly 2,000 candidate region proposals per image by hierarchically merging similar neighbouring image regions), warping each candidate region to a fixed size, a full CNN forward pass on each warped region independently, and a separate SVM classifier for each object class applied to the resulting CNN features.

Worked Example: Why R-CNN Was So Slow: Counting Forward Passes

Consider a single 1000×700 pixel image with Selective Search producing 2,000 region proposals, and a ResNet-50-scale backbone (Day 30 §4.3) taking roughly 50 milliseconds per forward pass on period-appropriate hardware.

R-CNN: 2,000 separate forward passes × 50ms ≈ 100 seconds per image — utterly impractical for any real-time or even near-real-time application.

Fast R-CNN (§4.2): exactly 1 forward pass on the full image (≈50ms), plus a cheap RoI Pooling operation per region — a roughly 100×-200× speedup from eliminating redundant backbone computation alone, before even accounting for other refinements.

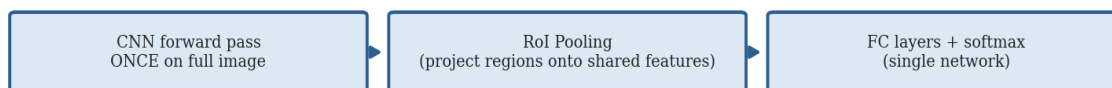
This single arithmetic comparison is why Fast R-CNN's "compute the backbone once, share it across all regions" insight (§4.2) was such a consequential improvement — it did not make the underlying CNN any faster, it simply stopped calling it thousands of times redundantly per image.

The R-CNN Family: From Slow Region-by-Region CNNs to a Fully Learned Pipeline

R-CNN (2014)



Fast R-CNN (2015)



Faster R-CNN (2015)



Figure 3. The R-CNN family's evolution: from running a full CNN separately on ~2,000 region proposals per image (R-CNN), to sharing computation across the whole image (Fast R-CNN), to learning the region proposals themselves (Faster R-CNN).

4.2 Fast R-CNN's Shared Feature Computation

R-CNN's central inefficiency is immediately visible in Figure 3's top row: running a full CNN forward pass separately for each of ~2,000 region proposals means the same underlying image is processed by the backbone thousands of times over, with enormous redundant computation across overlapping regions. Fast R-CNN (Girshick, 2015) fixed this directly: run the CNN backbone once on the entire image, producing a single shared feature map, and then use RoI (Region of Interest) Pooling to extract a fixed-size feature representation for each region proposal directly from this shared feature map, by projecting each proposal's coordinates onto the feature map and pooling within that projected region. This single change — share computation, do not repeat it per region — produced an order-of-magnitude speedup over R-CNN with no loss of accuracy, and directly parallels Day 28 §4.2's FLANN-versus-brute-force lesson: avoiding genuinely redundant computation, rather than making each individual computation faster, is often the larger lever.

4.3 Faster R-CNN's Learned Region Proposal Network

Fast R-CNN still depended on Selective Search — a classical, non-learned, comparatively slow algorithm — to generate region proposals in the first place, making it the pipeline's remaining bottleneck. Faster R-CNN (Ren et al., 2015) replaced Selective Search with a Region Proposal Network (RPN): a small, learned neural network, sharing the same backbone features Fast R-CNN's classification head uses, that directly predicts candidate object regions via a sliding-window classifier over a set of anchor boxes (Section 7.1) at every spatial position in the feature map. This made the entire detection pipeline — proposal generation and classification alike — a single, unified, fully learnable network for the first time, completing the same "hand-designed algorithm replaced by a learned one" trajectory Day 28 §7 traced for classical feature matching.

4.4 Why Two-Stage Detectors Trade Speed for Accuracy

The defining structural feature of every architecture in this section — and the reason they are called "two-stage" — is the clean separation between a proposal stage (find candidate regions that might contain an object) and a classification stage (decide what, if anything, each candidate region actually contains). This separation allows the classification stage to focus its full computational effort on a comparatively small number of promising candidates, rather than exhaustively evaluating every possible location and scale in the image — generally producing higher accuracy than one-stage alternatives (Sections 5–6), at the cost of the additional computation the proposal stage itself requires, and the inherent latency cost of running two sequential stages rather than one, a trade-off Section 8 develops in full using PP11's own comparison as a concrete illustration.

Use Case: Two-Stage Detection in Medical Screening

A radiology screening tool for detecting lung nodules in CT scans uses a Faster R-CNN-style two-stage detector (§4.3) rather than a faster one-stage alternative, since the application is offline (radiologist review, not real-time) and false negatives — a missed nodule — carry far higher cost than the extra processing time a two-stage pipeline requires.

This is a direct, concrete instance of Section 4.4's speed-for-accuracy trade-off playing out in a genuinely high-stakes domain, and a preview of the backbone and architecture considerations Day 37's medical imaging content develops in much greater depth for this exact class of application.

Model	Year	Region Proposals	Speed (relative)	Key Limitation Addressed
R-CNN	2014	Selective Search (classical)	Very slow	First to combine proposals + deep features
Fast R-CNN	2015	Selective Search (classical)	~10× faster than R-CNN	Shared backbone computation (§4.2)
Faster R-CNN	2015	Learned RPN (§4.3)	~10× faster than Fast R-CNN	Eliminated the remaining classical bottleneck

Research Spotlight: Anchor-Free Detectors

FCOS and CenterNet, briefly previewed here ahead of Day 32's deeper coverage, represent a research response to the anchor-box design complexity Section 7.2 described — rather than pre-defining a set of anchor shapes and matching objects to the closest one, these detectors predict object centres and sizes directly at each spatial position, removing the anchor-design hyperparameter search entirely.

This is conceptually a stepping stone toward DETR's (§10.1) even more complete removal of hand-designed detection machinery — first anchors go, then NMS goes too — illustrating a broader trend of progressively removing hand-engineered components from the detection pipeline, mirroring the classical-to-learned trajectory Day 26 §6 and Day 28 §7 traced for feature extraction specifically.

5 One-Stage Detectors: YOLO

5.1 YOLO's Core Insight: Detection as Single Regression

YOLO (You Only Look Once, Redmon et al., 2016) abandoned Section 4's two-stage proposal-then-classify structure entirely, reframing detection as a single regression problem solved in one forward pass: divide the image into a coarse grid, and have each grid cell directly predict, in one shot, the bounding boxes and class probabilities for any object whose centre falls within that cell — illustrated in Figure 4.

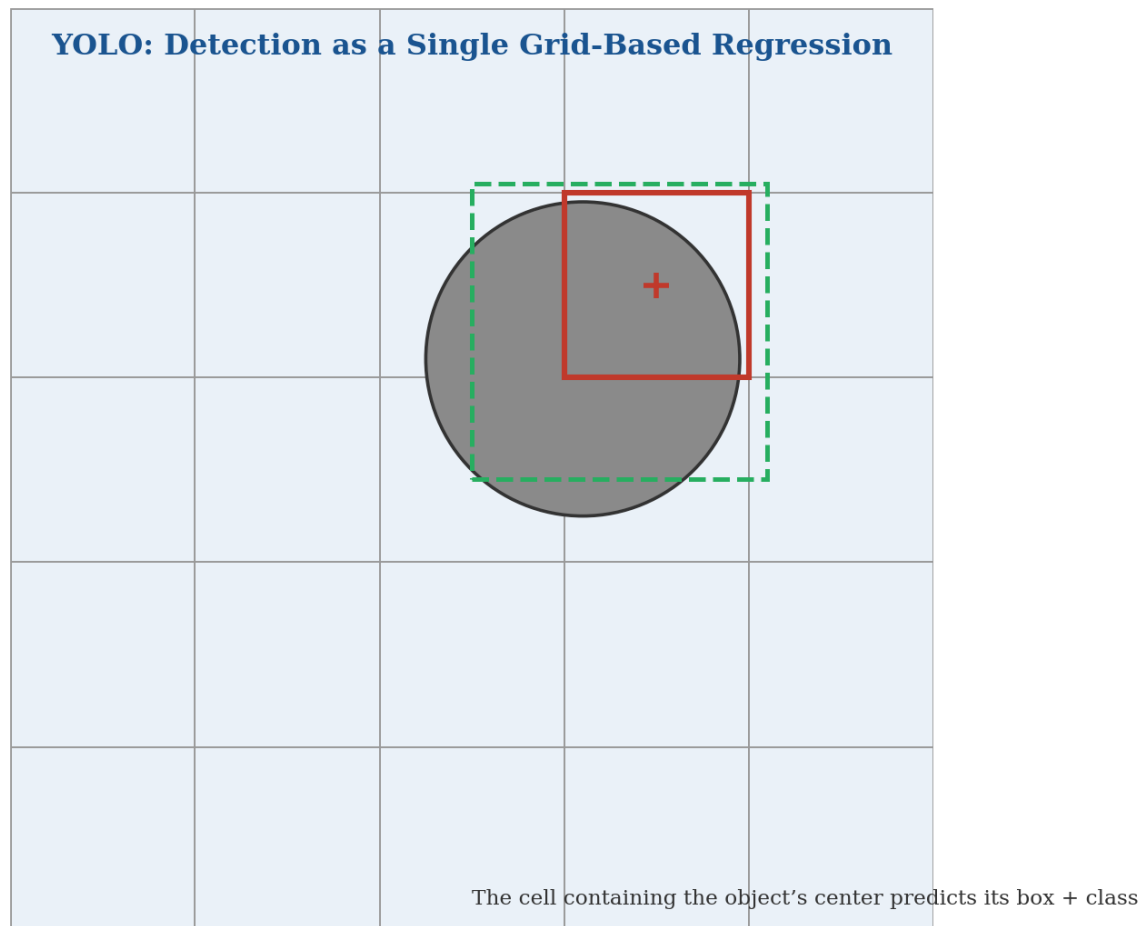


Figure 4. YOLO divides the image into a grid; the cell containing an object's centre is responsible for predicting that object's bounding box and class, all in a single forward pass.

5.2 Grid-Cell Prediction Mechanics

Each grid cell in YOLO's original design predicts a fixed number of candidate bounding boxes (each as a bounding box offset relative to the cell, plus a confidence score reflecting both "does this box contain an object" and "how accurate is this box's localization"), together with a single set of class probabilities shared across all boxes that cell predicts. This is a deliberately simple, direct mapping from image grid position to detection output — no separate proposal stage, no per-region CNN pass (Section 4.2's Fast R-CNN improvement notwithstanding, still a two-stage process); YOLO's entire detection output for an image is produced by a single forward pass through a single network, end to end.

5.3 Evolution Across YOLO Versions

YOLO has been refined across many subsequent versions since its 2016 introduction, and it is worth understanding the general architectural trend rather than an exhaustive version-by-version history, which is beyond this compendium's scope. Later versions progressively addressed the original design's weaknesses: anchor boxes (Section 7.1) were introduced to improve localization for objects with unusual aspect ratios; multi-scale prediction (conceptually similar to Section 6's SSD, covered next) was added to improve small-

object detection, an area the original single-grid-scale design struggled with; and backbone architectures were progressively updated to track Day 30's broader architectural advances (deeper, more efficient backbones as they became available). Across every version, the core "single-pass, grid-based regression" philosophy Section 5.1 introduced has remained the defining characteristic distinguishing the YOLO family from the two-stage R-CNN family.

Beyond these architectural refinements, later YOLO versions also progressively improved the loss function used during training — the original design used a relatively simple sum-of-squared-errors loss across box coordinates, objectness confidence, and class probabilities jointly, weighted by hand-tuned coefficients to balance these different objectives against each other. Later versions adopted more principled loss formulations, including IoU-based localisation losses (directly using Section 2.3's IoU metric, or refinements of it, as the training signal for box regression rather than raw coordinate error) and improved objectness/classification loss balancing, both contributing measurable accuracy gains independent of any change to the underlying grid-based architecture itself — a useful reminder that training objective refinements (Day 29 §7.2's subject, in the classification context) matter for detector accuracy just as much as architectural changes do.

5.4 Why YOLO Became the Real-Time Default

YOLO's single-forward-pass design makes it dramatically faster at inference than any two-stage detector, since there is no separate proposal stage to run sequentially before classification can begin — directly visible in Figure 6's speed comparison later in this day. This speed advantage is precisely why YOLO became the practical default choice for real-time detection applications — autonomous vehicle perception, live video analytics, robotics — where Section 4.4's two-stage accuracy advantage is a poor trade against a hard latency budget, echoing Day 30 §6.1's efficient-architecture motivation but applied to the detection head rather than the backbone alone. This speed-versus-accuracy positioning is exactly what PP11 asks you to verify directly, rather than take on faith, by fine-tuning and benchmarking both families yourself.

6 One-Stage Detectors: SSD

SSD and YOLO were published within months of each other in 2015–2016, both independently converging on the same fundamental single-shot philosophy (Section 5.1) while differing in this specific architectural detail — a useful illustration of a pattern that recurs throughout the history of computer vision architecture research (and, per Day 30 §2–§5, throughout backbone architecture research as well): once a genuinely good high-level idea (here, "predict everything in one pass") becomes apparent, multiple research groups often arrive at closely related but non-identical implementations of it around the same time, each making different specific engineering choices that later comparative work (Section 8) can then evaluate against each other directly.

6.1 SSD's Core Insight: Multi-Scale Feature Maps

SSD (Single Shot MultiBox Detector, Liu et al., 2016) shares YOLO's single-forward-pass philosophy (Section 5.1) but addresses a specific limitation the original YOLO design had: detecting objects across a

wide range of sizes using only a single, fixed-resolution grid. SSD instead makes predictions from multiple feature maps at different spatial resolutions within the same backbone — illustrated in Figure 5 — with each resolution naturally suited to detecting objects of a correspondingly different scale.

SSD: Detecting at Multiple Feature Map Scales Simultaneously

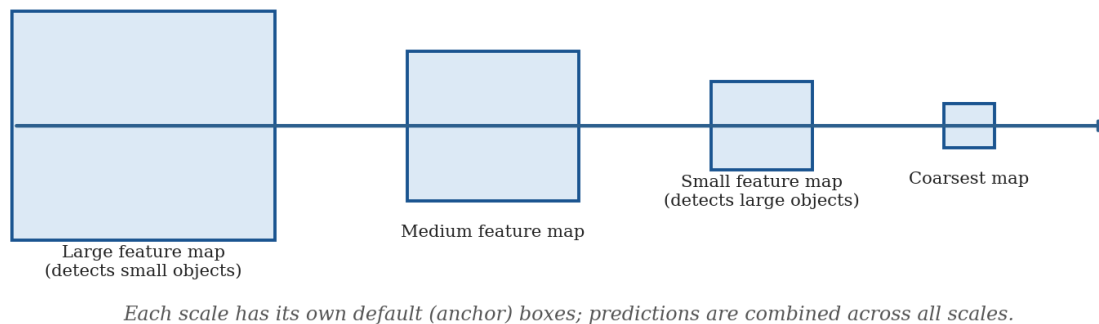


Figure 5. SSD predicts from multiple feature map resolutions simultaneously — larger, higher-resolution maps naturally suit smaller objects; smaller, coarser maps naturally suit larger objects.

6.2 Why Multi-Scale Detection Matters for Small Objects

The underlying reason multi-scale prediction helps small-object detection specifically connects directly to Day 29 §3.3's receptive field discussion: a feature map deep in the backbone (small spatial resolution, large receptive field per unit) has, by the time information reaches it, already been spatially downsampled many times over — a genuinely small object in the original image may occupy only a fraction of a single unit's receptive field at this depth, or may have been entirely lost through repeated pooling and striding (Day 29 §4). Predicting from an earlier, higher-resolution feature map, as SSD's multi-scale design does, preserves the spatial detail small objects need to be detected at all, while still allowing later, lower-resolution feature maps to handle large objects that a small, early receptive field could never fully capture.

Worked Example: Feature Map Resolution and Object Scale: A Concrete Illustration

Consider an SSD backbone producing feature maps at 38×38 , 19×19 , 10×10 , and 5×5 resolutions from a 300×300 input image (§6.1).

A small object (say, 20×20 pixels in the original image): at the 38×38 feature map, each cell corresponds to roughly 8×8 original pixels — the object spans multiple cells, giving the detector enough spatial resolution to localise it precisely.

The same small object at the 5×5 feature map: each cell now corresponds to roughly 60×60 original pixels — the entire small object could fall within a fraction of a single cell, with far too little spatial resolution remaining to localise it accurately.

This is the concrete, numeric version of Section 6.2's receptive-field argument: small objects need the spatial detail only early, high-resolution feature maps retain, which is exactly why SSD predicts from several resolutions simultaneously rather than relying on the backbone's final, most heavily downsampled feature map alone.

6.3 Default Boxes: SSD's Version of Anchors

At each spatial position within each of SSD's multiple feature maps, a set of default boxes (SSD's specific terminology for the more general anchor box concept, developed fully in Section 7.1) with varying aspect ratios and scales are placed, and the network predicts offsets and class probabilities relative to each default box rather than predicting raw, unconstrained box coordinates directly — a design choice shared with Faster R-CNN's RPN (Section 4.3) and later YOLO versions (Section 5.3) alike, underscoring that anchor/default boxes are a genuinely cross-cutting technique used across both detector families, not a feature unique to any one of them.

6.4 SSD vs. YOLO: A Direct Comparison

Since PP11 requires directly comparing these two specific detectors, it is worth summarising their structural similarities and differences explicitly. Both are single-shot, one-stage detectors (Section 5.1's philosophy, shared by both). Both use anchor/default boxes rather than predicting raw coordinates (Section 6.3). They differ primarily in how they handle multi-scale detection: SSD predicts from multiple feature map resolutions explicitly and simultaneously (Section 6.1); the original YOLO predicted from a single grid resolution, with multi-scale prediction added only in later versions (Section 5.3). This difference has historically made SSD somewhat stronger on small-object detection and YOLO somewhat faster at inference, though — as Section 8 discusses — later versions of both families have converged considerably, making an empirical comparison on your own specific dataset, exactly as PP11 requires, more informative than relying on these general historical tendencies alone.

Property	YOLO (original design)	SSD
Prediction resolution	Single grid resolution	Multiple feature map resolutions (§6.1)
Small-object handling	Historically weaker	Generally stronger, by design

Property	YOLO (original design)	SSD
Inference speed	Very fast	Fast, slightly more overhead than early YOLO
Anchor/default boxes	Added in later versions (§5.3)	Present from the original design (§6.3)
Backbone flexibility	Any backbone (Day 30 §9)	Any backbone (Day 30 §9)

7 Anchor Boxes & Non-Max Suppression

7.1 Anchor Boxes as Priors for Scale and Aspect Ratio

Anchor boxes are a set of pre-defined reference boxes, at fixed scales and aspect ratios, placed at every spatial position a detector considers — illustrated in Figure 6 for three example anchors at a single location. Rather than a detection head predicting a box's coordinates completely from scratch (a much harder regression problem, especially early in training when the network has no prior notion of what a "reasonable" box looks like), it instead predicts a small offset and scale adjustment relative to the nearest-matching anchor — a considerably easier learning problem, since the anchor already encodes a reasonable starting guess for the object's likely size and shape.

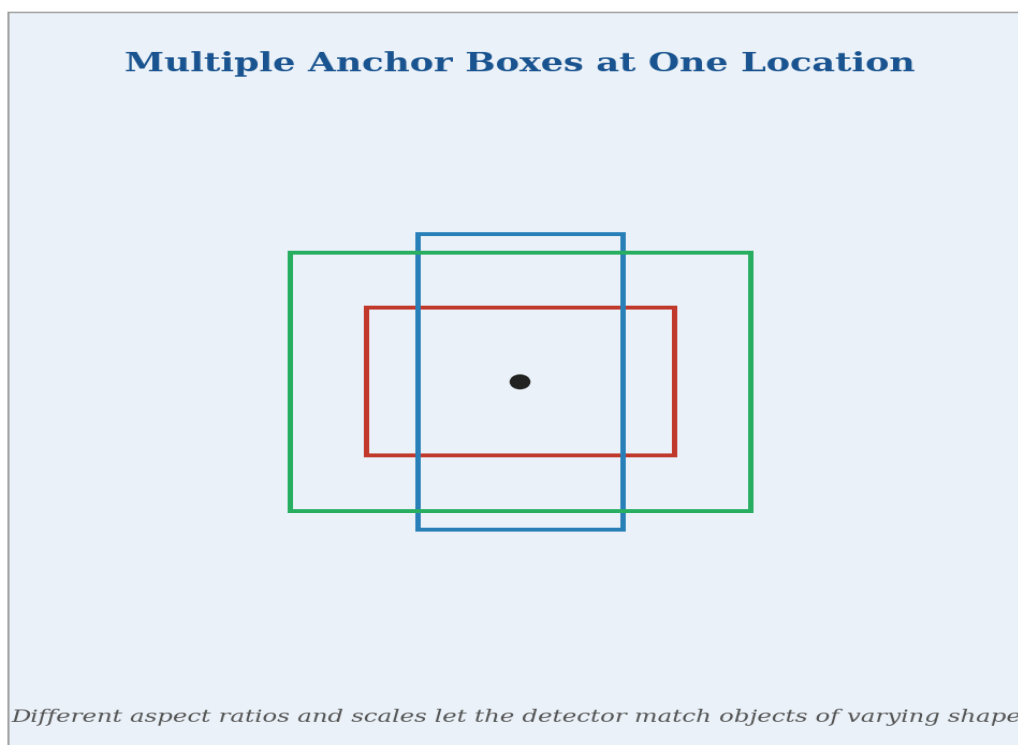


Figure 6. Multiple anchor boxes of different scales and aspect ratios are placed at every spatial location, giving the detector reasonable starting priors to refine rather than predicting box coordinates entirely from scratch.

Anchor boxes solve a genuine chicken-and-egg problem in detector training worth stating explicitly. Early in training, before the network has learned anything useful, a detection head predicting raw box coordinates from scratch has no reasonable starting point to refine — the regression target is, in effect, arbitrary from the network's perspective at initialisation. An anchor box gives every prediction a concrete, geometrically sensible starting point (a real box, of a real plausible scale and aspect ratio) to adjust from, converting the learning problem from "predict a box from nothing" into "predict a small correction to something already reasonable" — a substantially easier optimisation target, and a direct instance of the same "give the network a good prior to refine, not a blank slate" logic Day 29 §7.5's transfer learning discussion applied to network weights rather than box coordinates.

7.2 Anchor Design Trade-offs

The specific set of anchor scales and aspect ratios used is a genuine design choice with real consequences, not an arbitrary implementation detail. Too few anchor shapes, or anchor shapes poorly matched to the actual object shapes present in a dataset, force the detection head to learn large, difficult offset corrections for every object, hurting accuracy — this is precisely why some detection frameworks compute dataset-specific anchor shapes (via a clustering algorithm applied to the training set's ground-truth box dimensions) rather than using generic, dataset-agnostic default anchor shapes. Too many anchor shapes, conversely, increases computational cost and the number of predictions requiring Section 7.3's suppression step, without necessarily improving accuracy proportionally — a direct instance of Day 26 §1.1's inverse-optics-style "more parameters/priors is not automatically better" theme, here applied to detector design rather than to the ambiguity of the underlying vision problem itself.

Worked Example: Clustering Anchor Shapes from a Dataset

A common practical technique (used by several YOLO versions, §5.3) runs k-means clustering directly on the width and height of every ground-truth box in the training set, using $1 - \text{IoU}$ as the distance metric rather than ordinary Euclidean distance — producing a small set of anchor shapes (commonly 5–9) that closely match the actual object shapes present in that specific dataset.

For a dataset dominated by wide, short objects (vehicles photographed from the side, say), this clustering process would discover anchor shapes with correspondingly wide aspect ratios, rather than the generic, roughly square-to-moderately-rectangular anchor shapes a dataset-agnostic default set might use — directly improving the "reasonable starting guess" quality Section 7.1 described, and consequently reducing how large a correction the detection head must learn to predict for each object.

This is a genuinely worthwhile technique to consider for PP11 if your chosen dataset's object shapes differ noticeably from typical COCO-style natural photograph statistics — most modern YOLO implementations expose this anchor-clustering step as a configurable preprocessing option rather than requiring it to be implemented from scratch.

7.3 Non-Max Suppression: Mechanics

Because anchor boxes (Section 7.1) are placed densely across every spatial position and scale, a single real object very commonly triggers multiple overlapping, high-confidence detections — the raw, pre-suppression prediction set shown in Figure 7's left panel. Non-Max Suppression (NMS) resolves this

redundancy: sort all predictions by confidence score, select the highest-confidence prediction and keep it, then discard every remaining prediction whose IoU (Section 2.3) against the just-kept prediction exceeds a chosen threshold (commonly 0.5), and repeat this process with the remaining, un-discarded predictions until none are left — producing the clean, one-box-per-object result shown in Figure 7’s right panel.

Non-Max Suppression: Keeping the Highest-Confidence Box, Removing Redundant Overlaps

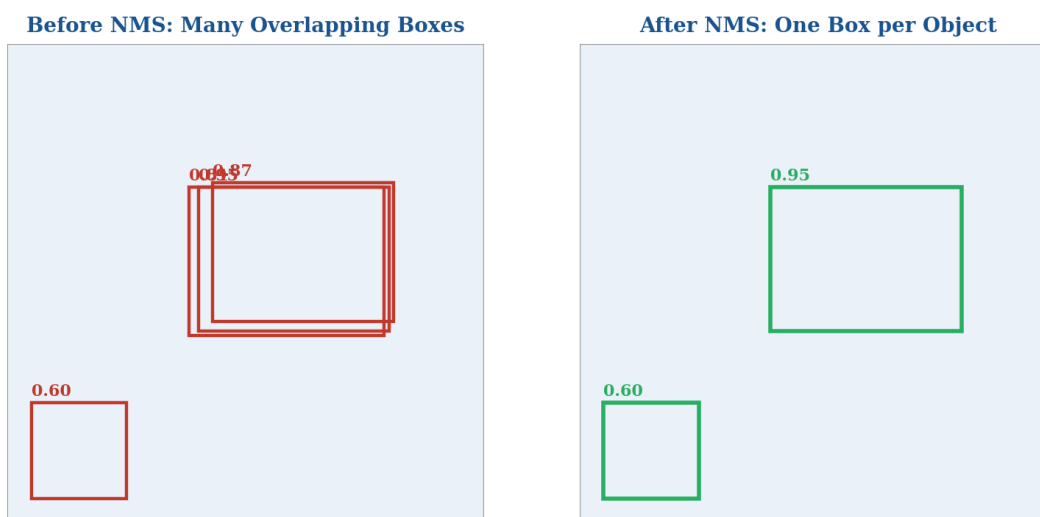


Figure 7. Non-Max Suppression removes redundant overlapping detections, keeping only the highest-confidence box for each genuine object — an essential post-processing step for every detector this day covers.

7.4 Soft-NMS: A Refinement

Standard NMS’s hard cutoff — a suppressed box is discarded entirely, regardless of how much genuine evidence supported it — can incorrectly remove a correct detection of a second, genuinely separate object that happens to overlap substantially with a first, higher-confidence detection (a common scenario in crowded scenes with many closely-spaced or partially-occluded objects). Soft-NMS addresses this by decaying a suppressed box’s confidence score proportionally to its overlap with the kept box, rather than discarding it outright — a box with very high overlap is suppressed nearly to zero confidence (behaving almost identically to standard NMS in the case standard NMS handles well), while a box with more moderate overlap retains a reduced but non-zero confidence, giving it a chance to survive a subsequent confidence-threshold filtering step if the evidence for it was otherwise strong. This section — anchor boxes and NMS together — underpins every detector in Sections 4 through 6, and is worth its own focused understanding independent of any single detector family, exactly the framing this section has taken.

```
def nms(boxes, scores, iou_threshold=0.5):
    # boxes: list of xyxy boxes; scores: matching confidence scores
    order = sorted(range(len(scores)), key=lambda i: scores[i], reverse=True)
    keep = []
    while order:
        current = order.pop(0)
        keep.append(current)
```

```
# discard remaining boxes with high IoU against the just-kept box (§2.3)
order = [i for i in order
          if compute_iou(boxes[current], boxes[i]) < iou_threshold]
return keep
```

8 Two-Stage vs. One-Stage: Tradeoffs

8.1 Synthesising Sections 4 Through 6

With two-stage detectors (Section 4) and one-stage detectors (Sections 5–6) both developed in full, this section synthesises the comparison directly, with particular depth on the YOLO-versus-SSD comparison specifically, since PP11 requires fine-tuning and directly comparing both.

It is worth stating this synthesis's core message plainly before the detailed tradeoff table: there is no universally "better" detector family among Sections 4 through 6's architectures, only a better fit for a specific combination of accuracy requirements, latency budget, and deployment constraints — precisely the same "no universal winner, only a better fit for specific constraints" conclusion Day 30 §9 reached for backbone architectures, now recurring at the level of detection-head design. This is not a hedge or a refusal to take a position; it is the central, empirically well-supported finding of the detection literature, and it is exactly what PP11's assignment is structured to have you rediscover directly through your own fine-tuning and benchmarking, rather than simply being told.

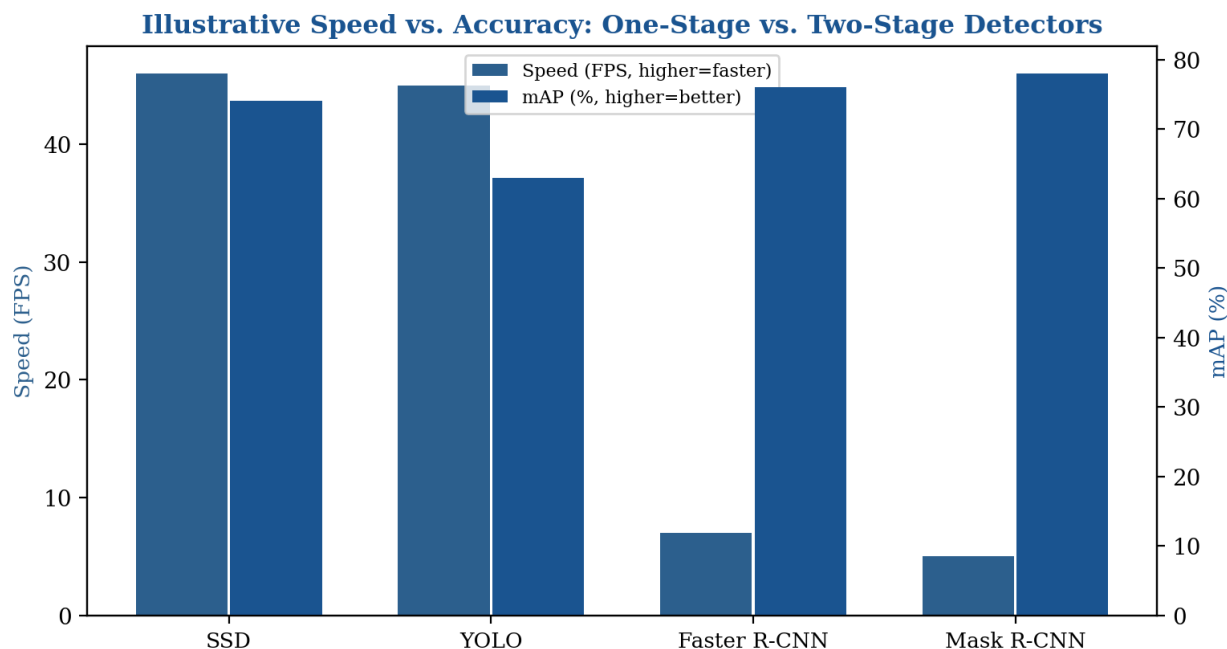


Figure 8. An illustrative speed-versus-accuracy comparison: one-stage detectors (SSD, YOLO) trade some accuracy for substantially faster inference than two-stage detectors (Faster R-CNN, Mask R-CNN).

Dimension	Two-Stage (R-CNN family)	One-Stage (YOLO/SSD)
Inference speed	Slower (sequential stages)	Faster (single forward pass)
Typical accuracy	Generally higher	Generally somewhat lower, gap narrowing
Training complexity	Higher (RPN + classifier)	Lower (single unified loss)
Small-object detection	Generally strong	Historically weaker; SSD/late YOLO improved this
Best fit	Accuracy-critical, offline processing	Real-time, latency-constrained deployment

It is also worth understanding, at a slightly deeper level than the summary table above, exactly where in the pipeline each family’s speed advantage or disadvantage actually originates, since this affects how much a specific optimisation (a lighter backbone, fewer anchors, a smaller input resolution) can be expected to help. A two-stage detector’s latency is dominated by two largely separate costs: the RPN’s proposal generation (Section 4.3) and the per-region classification stage that follows it, both of which scale, to varying degrees, with the number of proposals considered — reducing the number of proposals evaluated per image is therefore a direct, effective lever for speeding up a two-stage detector specifically. A one-stage detector’s latency, by contrast, is dominated almost entirely by the single backbone-plus-head forward pass — there is no separate proposal count to reduce, meaning the primary lever for speeding up a one-stage detector is backbone efficiency (Day 30 §6) or input resolution reduction, rather than any detection-specific parameter. This difference in where the computational cost actually lives is worth keeping in mind when optimising either family for PP11’s speed benchmark, since the most effective optimisation differs meaningfully between the two.

8.2 Expected Performance Differences

When one detector tends to outperform the other empirically, it is usually along one of a small number of predictable axes worth checking explicitly in PP11’s own results. Dense scenes with many small, closely-spaced objects tend to favour SSD’s explicit multi-scale design (Section 6.1) and, generally, two-stage detectors’ proposal-stage focus (Section 4.4) over YOLO’s historically coarser single-scale grid (Section 5.1—5.3). Sparse scenes with a small number of large, well-separated objects tend to show smaller accuracy differences between all detector families, since the localisation and small-object challenges that most differentiate them matter less. Inference speed differences are the most consistent and predictable axis of all — one-stage detectors should reliably outperform two-stage detectors on raw frames-per-second, regardless of the specific dataset, since this speed advantage follows directly from the architectural difference (Section 4.4) rather than from any dataset-specific property.

8.3 A Fair Head-to-Head Evaluation Methodology

A meaningful comparison between two detectors requires deliberate methodological care, directly relevant to PP11’s grading criteria. The same train/validation split must be used for both models — training YOLO

and SSD on different subsets of the same dataset would confound any observed accuracy difference with a difference in training data, making the comparison meaningless. The same IoU threshold convention must be used when computing mAP for both models (Section 3.4) — comparing YOLO's $\text{AP}@0.5$ against SSD's $\text{AP}@[0.5:0.95]$ would be comparing two different metrics, not two detectors on the same metric. And the same training budget (number of epochs, or a consistent early-stopping criterion) should be used for both, since a detector trained for twice as many epochs as its competitor has an unfair advantage unrelated to its underlying architecture. This methodology — same data, same metric, same training budget, differing only in the detector architecture itself — is exactly Day 30 §11.1's "fair architecture comparison" guidance, applied here specifically to detection.

8.4 Closing Perspective: What PP11 Should Reveal

It is worth stating explicitly, before moving on to this day's real-world applications, what a well-executed PP11 comparison should actually demonstrate. The assignment is not designed to produce a single "winner" between YOLO and SSD in any universal sense — both are mature, widely-deployed architectures, and Section 8's entire framework has argued that neither is universally superior. Instead, PP11 is designed to have you directly observe, on your own chosen dataset, the specific pattern of tradeoffs Sections 4 through 8 predicted: some difference in inference speed favouring one-stage detection generally (Section 5.4), some difference in small-object or dense-scene handling potentially favouring SSD's multi-scale design specifically (Section 6.1—6.2) if your dataset happens to contain many small or closely-spaced objects, and a training-time and implementation-complexity difference reflecting each architecture's underlying design. A strong PP11 submission is one that reports these observed differences honestly — including when they diverge from the general tendencies this day has described, which is a genuinely useful and expected outcome, not a sign of an error in your implementation — and explains them by reference to the specific properties of your dataset and the architectural mechanisms Sections 4 through 8 developed, rather than simply reporting a table of numbers with no interpretive discussion connecting them back to the underlying architecture.

9 Real-World Applications

Before working through the individual application areas below, it is worth noting a pattern that recurs across all four: in every case, the specific choice between a one-stage and a two-stage detector (Section 8) is driven by the same underlying latency-versus-accuracy calculus, just with different weights attached to each side depending on the application's specific stakes and constraints. This section is deliberately structured to make that recurring calculus visible across genuinely different domains, rather than presenting each application as an isolated case study.

9.1 Autonomous Driving Perception

Object detection is one of the core perception capabilities in autonomous vehicle systems, responsible for identifying and localising other vehicles, pedestrians, cyclists, and traffic signs and signals in real time from onboard cameras. The real-time latency requirement this application imposes is a direct, high-stakes

illustration of Section 5.4's YOLO-for-real-time positioning: a detector that is highly accurate but too slow to run within the vehicle's control loop's timing budget is not merely suboptimal, but potentially unsafe, making the speed-accuracy trade-off Section 8 developed a genuinely safety-relevant engineering decision in this domain, not merely a convenience consideration.

It is worth being specific about why this domain in particular tolerates so little latency. A vehicle travelling at highway speed covers a meaningful distance in even a small fraction of a second — at 100 km/h, a vehicle covers roughly 28 metres per second, meaning even a 200-millisecond perception delay corresponds to the vehicle having already travelled over 5 metres before a detected hazard is even registered by the system, before any braking or steering response can begin. This concrete calculation is why autonomous driving perception stacks are built almost universally around one-stage detectors (Section 5–6) or highly optimised, latency-tuned variants of two-stage detectors, rather than the largest, most accurate detectors available in the research literature — Section 8's speed-accuracy tradeoff is not an abstract concern in this domain, but a direct, quantifiable safety parameter.

9.2 Retail and Inventory Monitoring

Retail applications use object detection for automated checkout (detecting and identifying products in a shopping cart or on a scanning surface) and shelf-inventory monitoring (detecting empty or low-stock shelf sections from fixed or robot-mounted cameras) — the second of these is precisely the application Day 26 §9.1's retail shelf-monitoring callout box described, now revisited with the full technical detail (Sections 4–8) needed to understand exactly how such a system is actually built, rather than only why the business problem requires detection rather than classification.

It is worth noting a practical constraint retail detection systems face that distinguishes them from the autonomous-driving case (Section 9.1): retail environments are comparatively controlled (fixed camera positions, predictable lighting, a known and relatively stable set of product categories), which makes the small-object and dense-scene challenges Section 8.2 discussed less severe than in an open, unpredictable driving environment — a reasonable justification, in many deployments, for favouring inference cost and deployment simplicity (a smaller, more efficient one-stage detector, Day 30 §6's efficient-architecture backbone options included) over the last few percentage points of mAP a larger two-stage detector might offer.

9.3 Wildlife Camera-Trap Analysis

Conservation researchers deploy motion-triggered field cameras to monitor wildlife populations, and object detection automates what was previously an enormous manual review burden — automatically detecting and localising animals within each captured frame, filtering out the large volume of empty frames (triggered by wind, passing vegetation, or other false triggers) that camera traps typically produce. This is a direct extension of the individual-animal-matching application Day 28 §9.1's callout box described for classical feature matching — detection first locates an animal within a frame; the kind of individual-identification matching Day 28 described can then operate on the detected, cropped region specifically, illustrating how this module's techniques compose across days rather than existing as entirely separate, disconnected tools.

Beyond the three application areas already discussed, it is worth briefly cataloguing a few further domains where the exact same backbone-plus-head detection pattern (Section 1.2) is applied, to reinforce how general-purpose this day's content actually is. Agricultural monitoring uses detection to count and locate fruit for yield estimation, or to identify diseased plants for targeted intervention. Sports analytics uses detection to locate players and the ball across broadcast video frames, typically as the first stage of the tracking pipelines Day 32 develops in full. Manufacturing quality control uses detection to locate defects on a production line, often under exactly the tight latency constraints Section 9.1's autonomous-driving discussion described, since a defect-detection system integrated into a moving production line faces a genuinely comparable real-time processing requirement.

9.4 Security and Surveillance: An Ethical Callback

Security and surveillance applications — detecting and counting people, vehicles, or specific objects of interest in monitored spaces — raise directly the same ethical considerations Day 26 §9.2 introduced for facial recognition and Day 28 §9.4 extended to classical feature-based tracking. Object detection's capability to automatically locate and count people across a monitored space, entirely independent of any facial recognition or identity-matching component, still constitutes a meaningful surveillance capability in its own right — the same consent, proportionality, and demographic-fairness auditing considerations Day 26 §9.3's responsible-deployment framework laid out apply directly here, and are worth applying explicitly to any detection system deployed against people, not only to systems that additionally attempt to identify specific individuals.

A further, detection-specific ethical consideration worth naming explicitly: because a detector reports precise bounding-box locations rather than merely a whole-image label, detection-based surveillance systems can support considerably finer-grained behavioural analysis than classification alone could — counting how long a specific individual lingers in a specific location, tracking movement patterns across a monitored space, or flagging "unusual" spatial behaviour relative to a learned baseline. This finer-grained capability is a direct, technical consequence of detection's richer output structure (Section 1.1's comparison against classification), and it raises correspondingly sharper privacy and proportionality questions than a simple presence/absence classifier would — worth weighing explicitly against the genuine, legitimate use cases (Sections 9.1–9.3) this same technical capability also enables.

10 Research Frontiers

10.1 Transformer-Based Detection: DETR

DETR (Detection Transformer, Carion et al., Facebook AI, 2020) represents a genuinely different paradigm from every detector this day has covered: rather than anchor boxes (Section 7.1) and Non-Max Suppression (Section 7.3), DETR frames detection as a direct set-prediction problem, using a transformer (Day 30 §7's conceptual preview of the attention mechanism) to directly predict a fixed-size set of objects, with a learned bipartite matching between predictions and ground truth used during training to determine which prediction is responsible for which ground-truth object. This eliminates anchor boxes and NMS entirely — a network

trained this way learns not to produce redundant, overlapping predictions for the same object in the first place, rather than producing them and relying on a separate post-processing step (Section 7.3) to clean them up afterward.

It is worth being specific about what DETR's bipartite matching actually solves, since "eliminates NMS" alone understates the conceptual shift involved. Every detector in Sections 4 through 6 is trained with a many-to-one supervision signal at each spatial position or anchor — many anchors near a given object are all encouraged, to varying degrees, to predict that object, which is precisely why redundant, overlapping predictions arise in the first place and why Section 7.3's NMS post-processing step is necessary. DETR's bipartite matching instead computes a one-to-one assignment between the network's fixed set of output predictions and the ground-truth objects during training itself, using an optimal-assignment algorithm (the Hungarian algorithm, previewed here as a direct forward reference to Day 32's tracking content, which uses the same algorithm for a structurally similar assignment problem across video frames) — meaning the network is trained, from the start, to avoid producing redundant predictions for the same object, rather than being trained without that constraint and having redundancy cleaned up afterward.

A further practical note on DETR worth adding: while its removal of anchor boxes and NMS is architecturally elegant, DETR's original formulation was notably slower to train to convergence than the anchor-based detectors this day has otherwise covered, often requiring substantially more training epochs to reach comparable accuracy — a genuine practical cost of removing the anchor-based inductive bias (Section 7.1) that gives Sections 4–6's detectors a head start during training, echoing precisely the CNN-versus-ViT inductive-bias trade-off Day 30 §7.2 developed at length, now recurring in the detection-architecture context specifically. Later refinements of the DETR approach have substantially closed this training-efficiency gap, but it remains a useful, concrete illustration of a general principle worth carrying forward: removing a hand-designed component (anchors, NMS) trades away whatever helpful inductive bias that component provided, and the resulting architecture typically needs to recover that lost structure from data instead — which is not free, even when the architectural simplification itself is a genuine improvement.

10.2 Open-Vocabulary Detection

Every detector this day has covered is trained against a fixed, pre-defined set of object classes, decided in advance and baked into the network's final classification layer — precisely the same fixed-category limitation Day 26 §10.1 and Day 30 §10.3 identified for standard image classifiers, now applied to detection specifically. Open-vocabulary detection research, building directly on CLIP's joint image-text embedding space (previewed across Day 26 §10.1, Day 28 §10.4, and Day 30 §10.3), extends detection to arbitrary, natural-language-specified categories at inference time — detecting "a red bicycle" or "a person wearing a hard hat" without having been explicitly trained on bounding-box-labelled examples of either specific category, a genuinely significant capability extension directly built on the vision-language foundation this broader research programme has been previewing since Day 26.

10.3 Forward References: Day 32 and Day 33

This day's content is the direct, explicit foundation for the two days immediately following it. Day 32 extends object detection across time: tracking maintains a detected object's identity consistently across video frames, building directly on this day's per-frame detection output as its raw input signal — an object must first be detected in each frame (this day's subject) before it can be tracked across frames (Day 32's subject). Day 33 extends object detection to pixel-level precision: segmentation replaces this day's bounding boxes with precise per-pixel masks, and several of Day 33's architectures (Mask R-CNN prominent among them) are direct extensions of this day's Faster R-CNN (Section 4.3), adding a mask-prediction branch to an otherwise unchanged two-stage detection pipeline. Understanding this day's content thoroughly — the backbone-plus-head structure (Section 1.2), anchor boxes and NMS (Section 7), and the evaluation methodology (Section 3) — is a direct prerequisite for both of the days that follow.

10.4 Closing Perspective: The Detection Lineage in Context

It is worth situating this day's entire content within the module's broader arc before moving to the practical debugging checklist and assignment brief that close it out. Day 26 introduced the full computer vision task taxonomy, positioning detection as a specific, more demanding point along a spectrum from whole-image classification to pixel-precise segmentation. Days 27 through 30 built the necessary foundations — preprocessing, classical features, CNNs, and backbone architectures — that every detector in this day depends on directly, most visibly through Section 1.2's backbone-plus-head framing. This day has then developed detection's two major architectural lineages in full: the accuracy-focused, proposal-based two-stage family (Section 4) and the speed-focused, single-pass one-stage family (Sections 5–6), unified by the shared geometric and evaluation machinery (Sections 2–3, 7) both families depend on identically. Days 32 and 33, immediately following this one, each extend this day's output along a different axis — across time (tracking) and down to pixel precision (segmentation) — while depending on this day's content as their direct technical prerequisite, exactly as Section 10.3 described. PP11's hands-on comparison is where this entire chain of theory becomes concrete, measured, and genuinely your own finding rather than a fact taken on this compendium's authority.

11 Common Pitfalls: A Practical Debugging Checklist

Because detection combines a backbone (Day 30), a detection head (Sections 4–6), and a post-processing pipeline (Section 7), a poor result can trace back to any of several stages. The following checks catch the most common issues, and are directly relevant to debugging PP11.

- **Visualise predicted boxes on actual images before trusting any metric** — a mAP number alone can hide systematic problems (boxes consistently offset, wrong classes, boxes at the wrong scale) that are immediately obvious once a handful of predictions are drawn directly on their source images.
- **Confirm bounding box format conversions are correct in both directions** — per §2.1–§2.2, a silent `xyxy/xywh` mix-up produces boxes that are plausible-looking but systematically wrong, and will not raise any error — verify by plotting a few converted boxes against their source annotations before training begins.

- **Check for a mismatched IoU threshold between training-time anchor matching and evaluation-time scoring** — per §7.1–§7.2, some frameworks use a different IoU threshold for deciding which anchor is "responsible" for an object during training than the threshold used for evaluation (§3.1); this is not necessarily wrong, but is worth being aware of rather than assuming both are identical.
- **Verify NMS is actually being applied before inspecting raw predictions** — per §7.3, a detector's raw, pre-NMS output legitimately contains many redundant, overlapping boxes per object; do not mistake this expected raw output for a bug before confirming whether NMS has been applied yet.
- **Watch for class imbalance in the training set skewing per-class AP** — per §3.3, a class with very few training examples will very often show a much lower AP than a well-represented class, an effect worth checking for explicitly (via per-class AP, not only the averaged mAP) before concluding a detector architecture itself is weak at detecting that class.

Worked Example: A Complete Diagnostic Walkthrough

Symptom: PP11's YOLO model reports a validation mAP of only 0.12 after training, far lower than expected.

Step 1 (per §11): visualise predicted boxes on a handful of validation images. Suppose the boxes look roughly plausible in location but are all shifted noticeably to the right of their true objects.

Step 2: this consistent directional shift, rather than random scatter, points toward a coordinate system or format bug (§2.1–§2.2) rather than a genuine training failure — check the `xyxy/xywh` conversion code, and check whether image resizing (this section's earlier paragraph) is being applied to boxes consistently with the image itself.

Step 3: suppose the bug is found: the resizing rescale factor was computed from the image's original width but mistakenly applied to the box's y-coordinates too, rather than computing separate width and height scale factors. Fixing this single line and retraining brings mAP up to 0.61, in line with expectations for this dataset.

This walkthrough illustrates the general diagnostic pattern this section's checklist is built around: a single summary metric (mAP) rarely tells you where a problem lives, but a small amount of direct visual inspection, combined with recognising which specific failure pattern (consistent shift vs. random scatter vs. wrong classes vs. missing detections entirely) is present, very often localises the bug to a specific, fixable stage of the pipeline within minutes.

11.1 A Note on Reproducibility

Per Day 28 §11.1 and Day 29 §11.1's reproducibility guidance, detection training involves the same sources of randomness as classification training (weight initialisation, data augmentation, Day 27 §6) plus one further detection-specific source: anchor-to-ground-truth matching (Section 7.1—7.2) can, depending on the specific matching algorithm used, have a stochastic component sensitive to initialisation. For PP11's comparison specifically, fixing random seeds for both the YOLO and SSD training runs is worth doing explicitly, so that any observed difference between the two models reflects a genuine architectural difference rather than run-to-run training noise.

11.2 A Note on Comparing Against Published Benchmarks

It can be tempting, once PP11's own mAP numbers are computed, to compare them directly against published YOLO or SSD benchmark figures from the original papers or from leaderboards. This comparison is rarely meaningful without significant caveats, worth stating explicitly before drawing any conclusion from such a comparison. Published benchmarks are almost always computed on full, large-scale datasets (the complete COCO or Pascal VOC training and test sets) using extensive training budgets (hundreds of epochs, carefully tuned hyperparameters, often trained on hardware substantially more powerful than a single Colab T4 session provides) — none of which matches PP11's deliberately compact, time-constrained assignment scope. A PP11 mAP considerably lower than a published benchmark figure is the expected, normal outcome given these differences, not necessarily a sign of an implementation problem — the meaningful comparison for this assignment is always YOLO-versus-SSD under your own matched conditions (Section 8.3), not either model against its own published, differently-resourced benchmark result.

Glossary of Terms

This day has introduced a substantial amount of specialised vocabulary. The glossary below collects the most important terms in one place for quick reference, with a pointer back to the section where each is developed in full.

Term	Definition (see section for full detail)
Bounding box	A rectangle describing an object's location, in xyxy or xywh format (§2.1)
IoU (Intersection over Union)	A 0–1 score measuring overlap between two boxes (§2.3)
True positive (detection)	A prediction matching ground truth in both class and IoU threshold (§3.1)
AP (Average Precision)	The area under the precision-recall curve for one class (§3.2)
mAP (mean Average Precision)	AP averaged across all classes (§3.3)
Region proposal	A candidate image region that might contain an object (§4.1)
RoI Pooling	Extracting a fixed-size feature from a region on a shared feature map (§4.2)
Region Proposal Network (RPN)	A learned network predicting region proposals (§4.3)
Anchor box / default box	A pre-defined reference box a detector refines rather than predicting from scratch (§7.1)
Non-Max Suppression (NMS)	Removing redundant overlapping detections, keeping the highest-confidence box (§7.3)
One-stage / two-stage detector	Single unified prediction pass vs. separate proposal-then-classify stages (§4.4, §5.1)

Chapter Summary: Key Takeaways

- **Detection = backbone + detection head** (§1.2) — Day 30's entire backbone framework applies directly; what differs between detectors is the head.
- **IoU is the geometric primitive underlying everything in this day** (§2.3) — evaluation, anchor matching, and NMS all build on it.
- **mAP averages AP across classes; AP is the area under the precision-recall curve** (§3.2–3.3) — and the IoU threshold convention used (§3.4) must be stated explicitly and applied consistently for any fair comparison.
- **Two-stage detectors separate proposal from classification; one-stage detectors predict everything in a single pass** (§4.4, §5.1) — the fundamental architectural distinction driving every speed-vs-accuracy trade-off in this day.
- **Anchor boxes give a detector a reasonable starting prior to refine, rather than predicting coordinates from scratch** (§7.1) — used across both two-stage and one-stage families alike.
- **NMS removes redundant overlapping detections** (§7.3) — an essential post-processing step for every detector in this day, with Soft-NMS (§7.4) as a refinement for crowded scenes.
- **A fair YOLO-vs-SSD comparison requires matched data, metric, and training budget** (§8.3) — directly the methodology PP11 requires you to apply.
- **This day is the direct foundation for Day 32 (tracking) and Day 33 (segmentation)** (§10.3) — both extend this day's per-frame, per-box detection output along a different axis.

Assignment / Practical Project Brief

PP11: Object Detection with YOLO/SSD

★ GRADED ASSIGNMENT

This is a graded assignment, flagged explicitly as such because it carries a formal rubric (Section D below) rather than PP12's more open-ended design-choice framing (Day 26). The core requirement: fine-tune BOTH a YOLO model and an SSD model on the same custom or subset dataset — pretrained backbone through to fine-tuning, not merely running pretrained inference — and directly compare them using this day's Section 8.3 methodology.

A Core Requirements

Fine-tune both a YOLO model and an SSD model on the same train/validation split, starting from pretrained backbones (Day 29 §7.5, Day 30 §9.3) rather than random initialisation. This is a stricter bar than PP12 or PP19's single-model requirements, given this assignment's graded status — both detector families must be genuinely fine-tuned, not merely evaluated out of the box.

B Dataset

Choose a dataset appropriate for Colab T4 constraints — a compact subset of COCO (a handful of classes) or a small custom-labelled set are both reasonable choices, keeping fine-tuning tractable within a single 3-hour working session. Whichever dataset is chosen, verify its annotation format (Section 2.2) early, and convert it consistently to whatever format your chosen YOLO and SSD implementations each expect — a step worth completing and validating before any training begins, per Section 2.2's explicit warning.

Dataset Option	Format	Typical Size for PP11	Note
COCO subset (3–5 classes)	COCO JSON (§2.2)	1,000–3,000 images	Well-documented; large community tooling support
Pascal VOC subset	VOC XML (§2.2)	1,000–2,000 images	Simpler per-image XML structure than COCO
Custom labelled set	Whichever tool you annotate with exports	500–2,000 images	Most relevant to a specific application; more annotation effort

```
# Example: verifying and converting a COCO-format subset for both frameworks
from pycocotools.coco import COCO

coco = COCO("annotations/instances_train_subset.json")
cat_ids = coco.getCatIds(catNms=["person", "car", "bicycle"]) # pick a compact subset
img_ids = coco.getImgIds(catIds=cat_ids)
```



```
print(f"{len(img_ids)} images across {len(cat_ids)} classes")

# COCO boxes are (x_min, y_min, width, height) -- Section 2.2's hybrid format
# Convert to YOLO's normalised xywh (Section 2.1) before training a YOLO model:
def coco_to_yolo_box(box, img_w, img_h):
    x_min, y_min, w, h = box
    x_center = (x_min + w / 2) / img_w
    y_center = (y_min + h / 2) / img_h
    return x_center, y_center, w / img_w, h / img_h
```

Worked Example: Estimating Your Training Time Budget

Given a Colab T4 session's practical 3-hour working window, and needing to train two separate models (YOLO and SSD) plus leave time for evaluation and write-up, budget roughly 45–60 minutes of actual training time per model as a reasonable starting allocation.

A compact dataset (a few thousand images, 3–5 classes) fine-tuning from a pretrained checkpoint (Day 29 §7.5) typically reaches a reasonable, though not fully converged, accuracy within 15–30 epochs at this scale — use a small held-out validation subset to monitor the loss curve (Day 29 §11's debugging checklist) and stop early if validation loss plateaus well before your allocated time budget is exhausted, rather than training for a fixed epoch count regardless of whether it is still improving.

If either model is clearly still improving when your time budget runs out, this is worth noting explicitly in your written analysis (Section C) — an honest "both models were still improving when training was stopped due to time constraints" is a more defensible, more scientifically honest conclusion than silently presenting an under-trained comparison as if it were a fully converged one.

C Deliverables

- Two working fine-tuned pipelines (YOLO and SSD) on the same train/val split
- mAP evaluation for both models against the same ground truth, using a consistent IoU convention (Section 3.4) for both
- A direct comparison table/plot: mAP, training time, inference speed (FPS), and qualitative prediction differences on a handful of example images
- A short written analysis: which detector performed better on this dataset and why, explicitly tying the observed result back to Section 8's tradeoff discussion rather than reporting numbers alone

```
import time

# Measuring inference speed fairly -- same hardware, same batch size, same image count
def benchmark_fps(model, dataloader, n_batches=50):
    model.eval()
    start = time.time()
    with torch.no_grad():
        for i, (images, _) in enumerate(dataloader):
            if i >= n_batches:
                break
            _ = model(images.to("cuda"))
    elapsed = time.time() - start
    n_images = n_batches * dataloader.batch_size
    return n_images / elapsed # frames per second
```



```
yolo_fps = benchmark_fps(yolo_model, val_loader)
ssd_fps = benchmark_fps(ssd_model, val_loader)
```

D Grading Rubric

Rubric Component	Weight	What's Assessed
YOLO implementation correctness	20%	Correct fine-tuning setup; sensible training curve; reasonable predictions
SSD implementation correctness	20%	Correct fine-tuning setup; sensible training curve; reasonable predictions
Evaluation rigor	25%	Consistent methodology across both (§8.3): same split, same IoU convention, same training budget
Comparison & analysis quality	20%	mAP, speed, and qualitative comparison; analysis grounded in §8's tradeoffs, not just numbers reported
Code quality & write-up	15%	Clean, documented code; clear, well-organised written analysis

It is worth reading this rubric's weighting deliberately: evaluation rigor and comparison quality together account for 45% of the total grade — nearly half — reflecting this assignment's core purpose. PP11 is not primarily an exercise in getting two detectors to train successfully (worthwhile, but only 40% combined); it is primarily an exercise in producing a genuinely fair, well-reasoned comparison between them (Section 8.3's methodology), and the rubric is weighted accordingly. A submission with two technically correct but sloppily-compared models will score notably lower than one with two adequately-trained models compared with real methodological care.

E Tips and Common Pitfalls

- **IoU threshold mismatches between the two evaluations** — per §3.4 and §8.3, verify both models are evaluated with the identical IoU convention; comparing YOLO's $AP@0.5$ against SSD's $AP@[0.5:0.95]$ invalidates the comparison entirely.
- **Inconsistent train/val splits** — per §8.3, generate the split once and reuse the identical file list for both models; regenerating a "random" split separately for each model risks subtle data leakage or an unfair comparison.
- **NMS misconfiguration** — per §7.3–§7.4, using very different NMS IoU thresholds or confidence cutoffs between the two models during evaluation can make a genuinely comparable pair of detectors look artificially different; keep post-processing settings matched unless a difference is a deliberate, documented part of the comparison.
- **GPU memory limits when fine-tuning two models in one Colab session** — restart the runtime (or explicitly free GPU memory) between training YOLO and SSD if both are trained in the same notebook session, to avoid out-of-memory errors from residual allocated tensors.

Beyond the checklist below, it is worth re-reading Section 8.4's closing perspective once your training and evaluation are complete, before writing the final analysis section — a strong PP11 submission connects its specific, measured results back to the architectural reasoning Sections 4 through 8 developed, rather than presenting numbers in isolation.

F Submission Checklist

- YOLO fine-tuned and evaluated with mAP on the held-out validation split
- SSD fine-tuned and evaluated with mAP, using the identical split and IoU convention
- Inference speed (FPS) benchmarked for both models under matched conditions
- Comparison table/plot completed with your own measured numbers, not placeholders
- Written analysis explicitly connects your results to Section 8's speed/accuracy tradeoff discussion
- Code is documented, and a short README explains how to reproduce your results

End of Day 31 — ready for gap review and expansion into full compendium.